

UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA Nº 09

NOMBRE COMPLETO: José Eduardo Martínez García

Nº de Cuenta: 320304679

GRUPO DE LABORATORIO: 13

GRUPO DE TEORÍA: 05

SEMESTRE 2027-1

FECHA DE ENTREGA LÍMITE: 28/10/2026

CALIFICACIÓN: _____

Reporte de práctica 9-1: Animación Básica

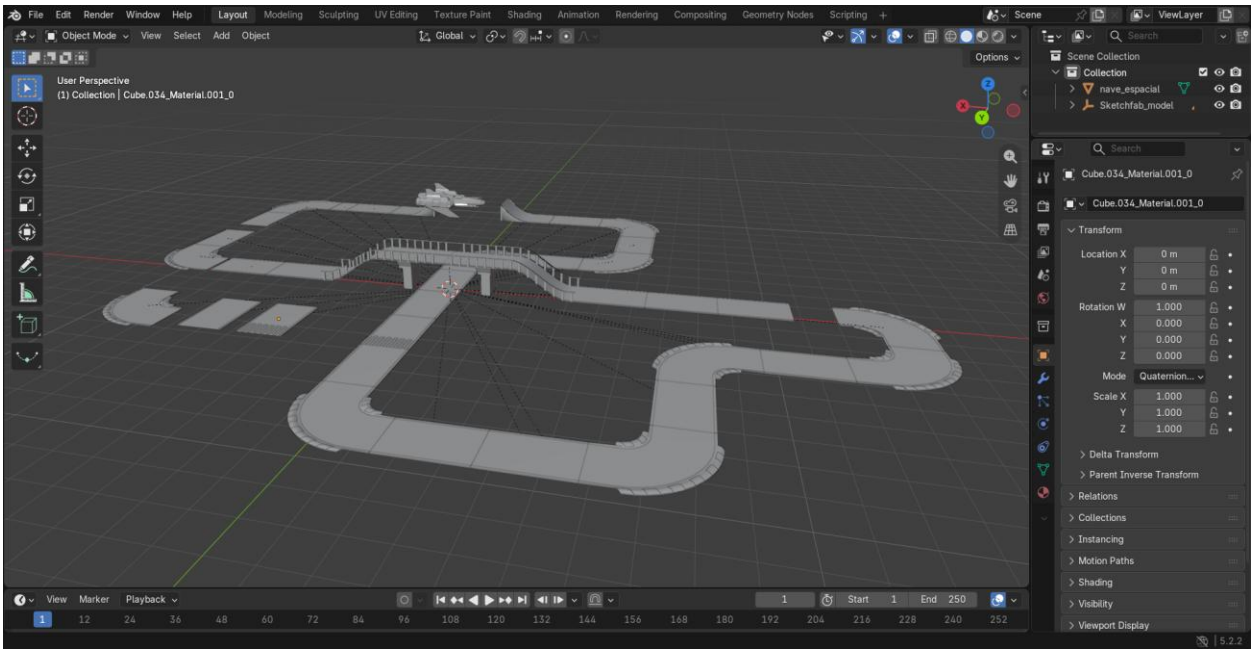
EJERCICIOS DE SESIÓN:

1. Actividades realizadas

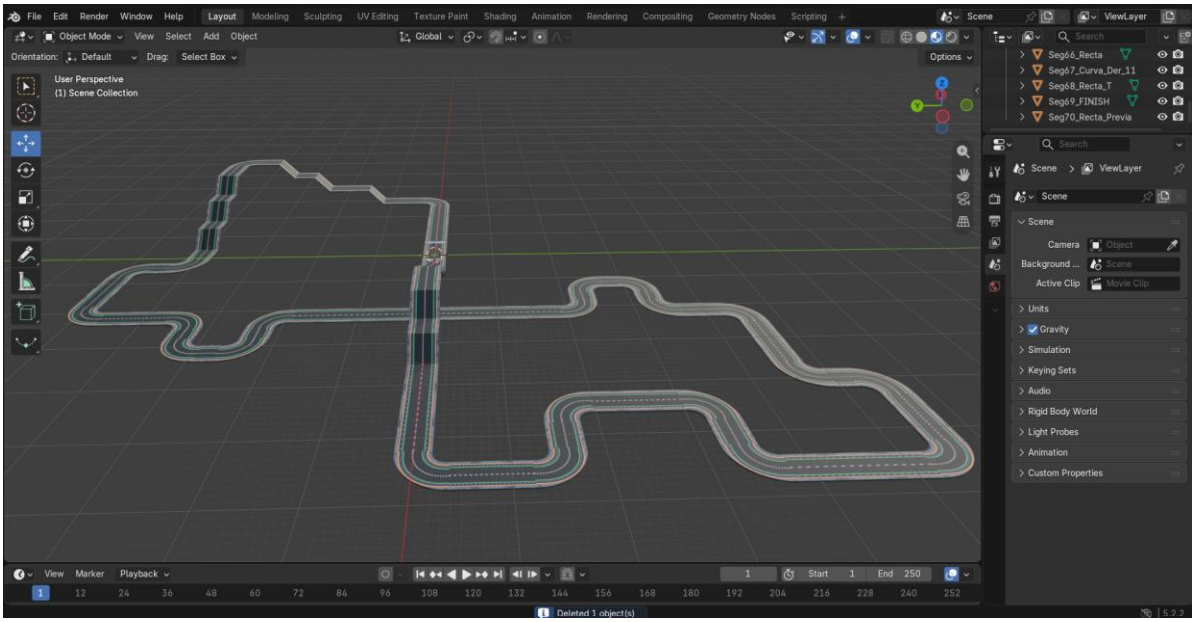
En esta práctica metí dos cosas nuevas a la escena de la práctica pasada: una nave espacial que da la vuelta completa a un circuito de carreras (con rectas, curvas, rampas de subida y de bajada) y un satélite que vuela solo por una trayectoria y mueve sus partes automáticamente, cuando antes todo eso lo hacía con el teclado. Al rover y al avión no les cambié nada, y tampoco escalé la nave ni el escenario; lo único que escalé fue la pista, para que cupiera en el piso de 600 × 600.

a) La pista de carreras

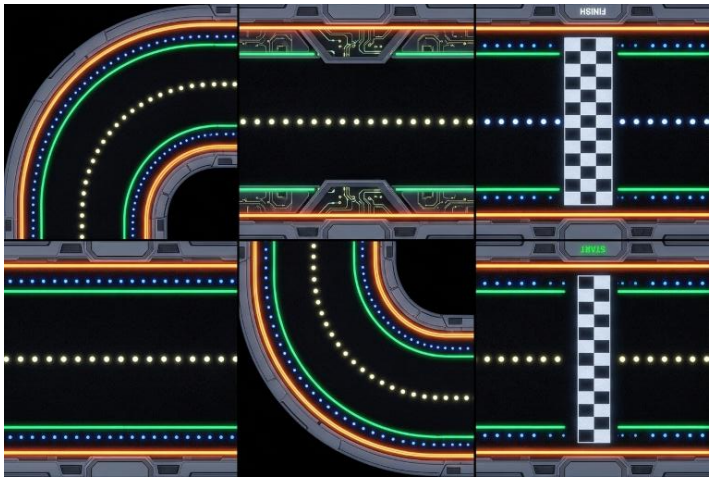
La pista la tomé de un modelo que encontré en internet, pero era mucho más chica y no me servía, así que la rehice completa en Blender.



La armé por secciones, usando cubos y figuras básicas, igual que las piezas de la pista original, y le puse una textura distinta a cada tipo de sección (salida, meta, recta, recta tecnológica, curva izquierda, curva derecha y la base). El resultado son 70 secciones: tiene rampas a 45°, mesetas, una parte alta con una curva arriba y un cruce en forma de 8 donde el circuito pasa por debajo de una meseta. Las curvas son de 90° con radio 38.



Se utizaron las siguientes texturas para poder poner las secciones del código:



En el código la pista es un modelo más y se carga con el resto, nada más la escalo a la mitad y la acomodo en el centro del piso:

```
const float PISTA_ESCALA = 0.5f;
...
pista_carreras = Model();
pista_carreras.LoadModel(PISTA_MODELO);
```

b) Los 70 tramos de la pista (puntos coordenados)

Para que la nave sepa por dónde ir, dividí la pista en sus 70 secciones que utilice para armas la pista y, para cada una, guardé en una tabla dónde empieza, cuánto mide, el punto inicial y final, hacia dónde mira y si es recta o curva. A la IA solamente le pedí ayuda para encontrar los puntos

coordenados de esas 70 secciones; con esos datos yo armé la tabla en el código. Aquí pongo nada más las primeras líneas porque son muchísimas(70):

```
enum TipoTramoNave { RECTA, CURVA };
struct TramoNave { float inicio, longitud; glm::vec3 desde, hasta;
                glm::vec2 rumbo; int sentido; TipoTramoNave tipo; };

const TramoNave TRAMOS_NAVE[] = {
 { 0.000f,  40.000f, vec3(0,0,0),      vec3(-40,0,0),        vec2(-1,0), 0, RECTA }, // Seg01_START
 { 40.000f, 40.000f, vec3(-40,0,0),    vec3(-80,0,0),        vec2(-1,0), 0, RECTA }, // Seg02_Recta
 { 80.000f, 40.000f, vec3(-80,0,0),    vec3(-108.28,28.28,0),vec2(-1,0), 0, RECTA }, // Seg03_Rampa_Sube_1
 ...
 { 568.860f, 59.690f, vec3(-521.99,0,0.0073), vec3(-559.99,0,37.99),
                 vec2(-0.9997,0.0248), -1, CURVA }, // Seg11_Curva_Izq_1
 ... (hasta el tramo 70)
};
```

Cada renglón es una sección: "inicio" es la distancia acumulada desde la salida, "sentido" vale −1 si la curva es hacia la izquierda y +1 si es hacia la derecha, y en las rectas vale 0. Con esto la nave puede saber en qué sección va solo con la distancia que lleva recorrida.

c) Funciones de recta y curva (base del movimiento)

Las funciones de recta y de curva las hice yo, basándome en las que ya había definido en el código para el avión y el rover. Las del avión son estas: en la recta, con d la distancia recorrida desde el punto de inicio (x0, z0) y yaw0 el ángulo con el que arranca, la posición es $x = x0 + \sin(yaw0)\cdot d$ y $z = z0 + \cos(yaw0)\cdot d$. En la curva, con radio R = 40, el ángulo que va girando es $\theta = d / R$, el nuevo ángulo es $yaw = yaw0 + \theta$ y la posición es $x = x0 + R(\cos yaw0 - \cos yaw)$, $z = z0 + R(\sin yaw - \sin yaw0)$. El rover es más fácil, solo va en línea recta: movCoche suma o resta movOffset·deltaTime según vaya o regrese.

```
//Avion: recta y curva con radio R
x = x0 + sin(yaw0) * d;
z = z0 + cos(yaw0) * d;

yaw = yaw0 + d / R;
x = x0 + R * (cos(yaw0) - cos(yaw));
z = z0 + R * (sin(yaw) - sin(yaw0));

//Rover: recta de ida y vuelta
movCoche += movOffset * deltaTime;   // ida
movCoche -= movOffset * deltaTime;   // regreso
```

Para la pista usé la misma idea. Si el tramo es recta, la posición es una mezcla entre el punto inicial y el final: P = mix(desde, hasta, u), donde u va de 0 a 1 dentro del tramo. Si es curva, la curva es un cuarto de círculo (90°): primero saco el centro, que es el punto medio de la cuerda (la línea que une el inicio con el final) más la mitad de la perpendicular de la cuerda por el sentido; luego el ángulo es $sentido\cdot u\cdot \pi/2$ y giro el vector que va del centro al punto inicial ese ángulo. La posición es el centro más ese vector girado.

```
glm::vec3 puntoPista(float distanciaMundo)
{
   float s = std::fmod(distanciaMundo / PISTA_ESCALA, LONGITUD_CIRCUITO);
   ... //se busca el tramo donde cae s y u = (s - inicio) / longitud

   //RECTA
   local = glm::mix(desde, hasta, u);

   //CURVA (cuarto de circulo)
   centro = (a + b) * 0.5f + signo * glm::vec2(-cuerda.y, cuerda.x) * 0.5f;
   angulo = signo * u * (PI_F * 0.5f);
   radial = rotate(radioInicial, angulo);
   local  = (centro + radial, mix(alturaInicial, alturaFinal, u));

   return PISTA_OFFSET + PISTA_ESCALA * local;
}
```

Resumiendo: la nave se mueve con una sola cosa, la distancia recorrida s, y puntoPista(s) me regresa dónde debe estar en la pista (recta con mix, curva con un arco de radio 38).

d) Movimiento de la nave sobre la pista

Cada frame le sumo a s la velocidad por deltaTime. Para saber hacia dónde mira, saco el punto de la pista un poco adelante y otro un poco atrás de la nave y con la línea entre ellos obtengo la dirección. Con eso la nave se inclina en las rampas y gira en las curvas. Además la subo lo necesario para que el casco no se meta en las rampas (reviso varios puntos a lo largo de la nave). Las luces y el fuego los dejé colgados de la misma matriz, así que la jerarquía de los propulsores se mantiene.

```
void actualizaNave(float dt)
{
   naveS = fmod(naveS + NAVE_VELOCIDAD * dt, longitudMundo);

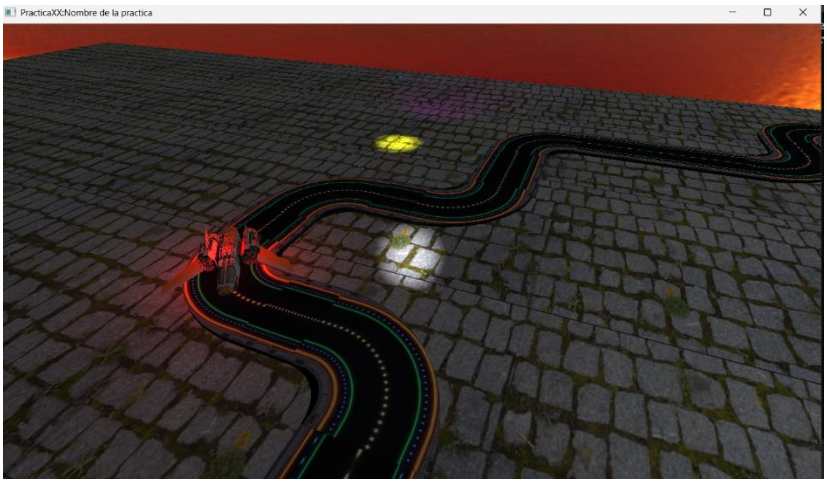
   centro = puntoPista(naveS);
   frente = puntoPista(naveS + NAVE_SEMILARGO);
   atras  = puntoPista(naveS - NAVE_SEMILARGO);

   naveAdelante = normalize(frente - atras);
   naveDerecha  = normalize(cross(naveAdelante, vec3(0,1,0)));
   naveArriba   = normalize(cross(naveDerecha, naveAdelante));

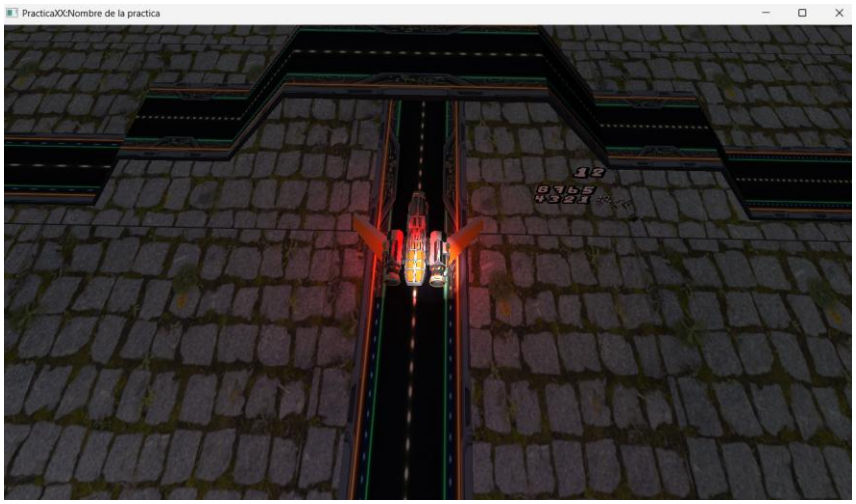
   //holgura: se revisan 7 puntos del casco (k = -3..3)
   navePos = centro + naveArriba * (NAVE_ALTURA_FLOTACION + holgura);

   modelNaveAux = mat4(NAVE_NARIZ_SIGNO * naveAdelante, naveArriba,
                       NAVE_NARIZ_SIGNO * naveDerecha, navePos);
   modelNaveAux = rotate(modelNaveAux, PI_F, vec3(0,1,0));
   modelNave    = scale(modelNaveAux, NAVE_ESCALA);
}
```

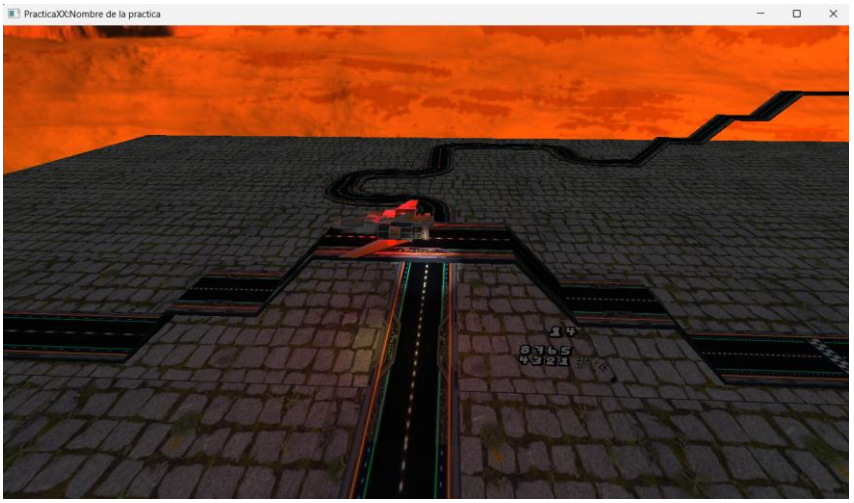
Nave en una curva:



Nave en una parte directa de la trayectoria de la pista:



Nave sobre una meseta de pista:



e) Trayectoria del satélite

El satélite sigue una trayectoria parecida a la del avión: arranca en el borde del piso, va en línea recta, hace una vuelta completa de 360° con un radio grande (120) centrada en el centro de la escena y sigue recto hasta el otro lado. Todo depende de la distancia recorrida d. En la vuelta, el ángulo es a = (d − largoRecta) / radio, y la posición es x = R·sin(a), z = R·cos(a), con el satélite mirando hacia a.

```
void trayectoriaSatelite(float d)
{
   float largoRecta  = SAT_LIMITE;
   float largoVuelta = 2.0f * PI_F * SAT_RADIO;

   if (d < largoRecta)                    //recta inicial
   {   satX = -SAT_LIMITE + d;  satZ = SAT_RADIO;  satYaw = 0.0f; }
   else if (d < largoRecta + largoVuelta)    //vuelta de 360
   {   float a = (d - largoRecta) / SAT_RADIO;
      satX = SAT_RADIO * sin(a);
      satZ = SAT_RADIO * cos(a);
      satYaw = a; }
   else                                 //recta final
   {   satX = d - largoRecta - largoVuelta;  satZ = SAT_RADIO;  satYaw = 0.0f; }
}
```

Cuando llega a un extremo da un giro de 180° suavizado y regresa por el mismo camino, igual que el rover, usando una pequeña máquina de 4 fases (ida, giro, regreso, giro).

```
case 0: satDistancia += SAT_VELOCIDAD * dt; ...   //ida
case 1: satProgresoGiro += dt / SAT_TIEMPO_GIRO; ... //giro 180
case 2: satDistancia -= SAT_VELOCIDAD * dt; ...   //regreso
default: satProgresoGiro += dt / SAT_TIEMPO_GIRO; ... //giro 180

float suave = satProgresoGiro * satProgresoGiro * (3.0f - 2.0f * satProgresoGiro);
```

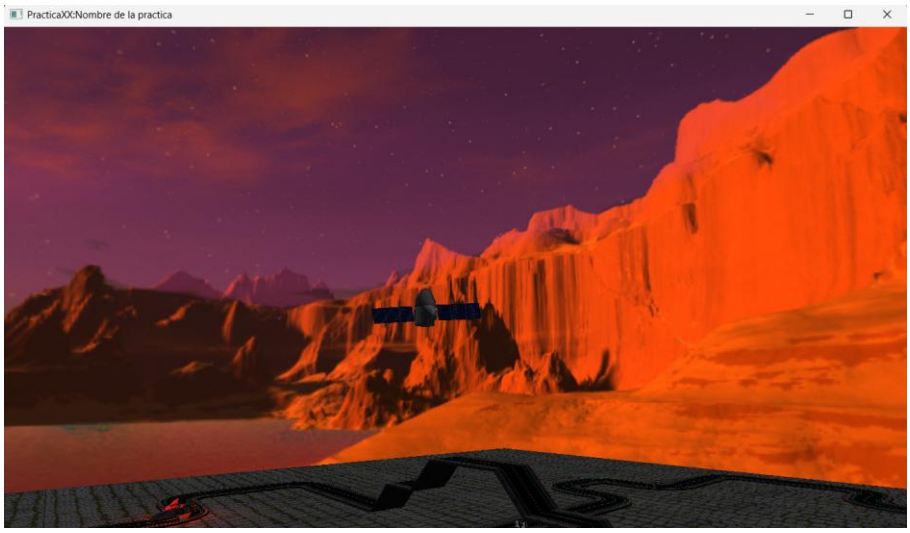
f) Articulaciones automáticas del satélite

Lo que antes movía con teclas ahora se mueve solo en el mismo actualizaSatelite: el cuerpo de arriba da vueltas sin parar y los paneles se abren y cierran de −45° a 45°, cambiando de sentido cuando llegan al límite (los límites los puse yo).

```
satAngSup += SAT_SUP_VELOCIDAD * dt;
if (satAngSup > 360.0f) satAngSup -= 360.0f;

satAngAla += satSentidoAla * SAT_ALA_VELOCIDAD * dt;
if (satAngAla >= SAT_ALA_MAX)      { satAngAla = SAT_ALA_MAX; satSentidoAla = -1.0f; }
else if (satAngAla <= SAT_ALA_MIN) { satAngAla = SAT_ALA_MIN; satSentidoAla =  1.0f; }
```

Satelita sobre su trayectoria:



g) Luces

Mantuve las luces de prácticas pasadas: las del avión, el rover y las de la nave (luz puntual, faro y luces de los motores, además del fuego con las teclas X y C, Z para el faro de la nave y 1 para cambiar el color del faro). Quité las luces que ya no se usaban y dejé las de los modelos: 3 puntuales, 5 spot y la direccional. Para que no truene, puse static_assert para que el máximo de luces del código sea el mismo que el del shader.

```
static_assert(MAX_POINT_LIGHTS == 6, "debe coincidir con shader_light.frag / CommonValues.h");
static_assert(MAX_SPOT_LIGHTS  == 7, "debe coincidir con shader_light.frag / CommonValues.h");
```

2. Problemas presentados

- Encontrar las funciones de movimiento: lo más difícil fue sacar las funciones para que cada objeto se moviera como yo quería. Lo resolví con la misma lógica del avión y del rover: curvas con un radio definido (el de la curva o apertura) y movimiento rectilíneo para las rectas, como en el rover.
- Las luces: quise mantener las funciones de luces de prácticas pasadas (por ejemplo las de la nave) y al volver a meter la lógica en practica9full.cpp me daba errores. Al final eran errores de definición de las teclas que usaba para modificar las luces, tanto las del avión como el fuego y las luces de la nave. Corregí los botones y todo volvió a funcionar. También hubo que revisar que el máximo de luces coincidiera con el shader.
- La pista era muy chica y sin variedad: la que encontré en internet no servía, así que la rehice y la texturicé por secciones. Luego la alargué para que diera más vueltas, con un cruce en 8 donde pasa por debajo de una meseta, y le agregué otra parte alta con una curva arriba.
- Escala de la pista: el modelo original era demasiado grande para el escenario de OpenGL (600 × 600 unidades). Escalé la pista (a la mitad) sin tocar el piso, ni la nave ni nada más, y la centré en el piso.
- Carga de texturas: tuve que corregir las rutas del archivo .mtl para que OpenGL reconociera las texturas de la pista. Además mi clase de modelos carga los .png como RGBA, así que convertí las texturas y agregué una textura base para el costado de la pista.

- Posición y orientación de la nave: aparecía al costado de la pista y quería que girara e inclinara natural en curvas y rampas, manteniendo la jerarquía de sus propulsores. Lo arreglé con la orientación por la línea entre un punto de adelante y uno de atrás, y ajustando hacia dónde apunta la nariz (la giré 180°). También se atravesaba en las rampas, así que la subí con la holgura del casco.
- Trayectoria del satélite: pensar cómo darle una vuelta de 360° centrada en la escena sin salirme del piso, lo resolví con el mismo estilo de recta–curva–recta del avión.

3. Conclusión

a) Los ejercicios de la clase

Me parecieron de complejidad media-alta. Lo más pesado fue la pista: dividirla en secciones, texturizarla y lograr que la nave la siguiera con la inclinación correcta. El satélite fue más sencillo porque reutilicé la idea del avión. Ya teniendo claro que todo depende de una distancia recorrida, las rectas y curvas salieron más fácil.

b) Comentarios generales

Creo que faltó explicar más a detalle cómo sacar una trayectoria sobre un modelo con rampas y cómo orientar un objeto sobre ella, y ir más lento con las luces, que sigue siendo lo que más se me complica. Una sugerencia sería ver un ejemplo corto de un objeto siguiendo una pista con pendientes antes de la práctica.

c) Conclusión

Al final logré que la nave recorra toda la pista, subiendo, bajando y girando en las curvas, y que el satélite vuele solo con sus partes en movimiento, manteniendo lo de las prácticas pasadas. Aprendí que con unas pocas funciones básicas (recta con mix, curva con un arco de radio fijo y una distancia recorrida) se pueden hacer movimientos bastante complejos.

Bibliografía

Blender Foundation. (2024). Blender manual. https://docs.blender.org/manual/en/latest/

de Vries, J. (2020). Learn OpenGL. https://learnopengl.com/

G-Truc Creation. (2024). OpenGL Mathematics (GLM) documentation. https://glm.g-truc.net/

Open Asset Import Library. (2024). Assimp documentation. https://assimp-docs.readthedocs.io/

Khronos Group. (2024). OpenGL reference pages. https://registry.khronos.org/OpenGL-Refpages/